Building a Native MCP Server with @modelcontextprotocol/sdk (TypeScript) + Neo4j v5: Production API Reference

TL;DR

- **Use the low-level `Server` class from `@modelcontextprotocol/sdk` v1.29.0 (latest stable; v2 is pre-alpha, stable release anticipated Q1 2026)** with `setRequestHandler(ListToolsRequestSchema, ...)` + `StdioServerTransport` for your Kabbalistic hypermetagraph server — this is the right primitive for a fixed, hand-authored tool set (`resolve_coordinate` , `find_intersections` , `traverse_lineage` , `semantic_search`) plus custom-URI resources, and it remains fully supported in 1.x.
- **Pair it with `neo4j-driver` v5 using `executeRead` / `executeWrite` managed transactions, the `neo4j+s://` (routing + TLS, for Aura) or `bolt+s://` (direct + TLS) scheme, and a `CREATE VECTOR INDEX ... OPTIONS {indexConfig: { vector.dimensions : N, vector.similarity_function : 'cosine'}}` index** for `semantic_search` .
- **The single biggest 0.x→1.x breaking change that will bite you is import paths**: every SDK subpath now requires the `.js` extension (`@modelcontextprotocol/sdk/server/index.js`) under ESM `NodeNext` / `Node16` resolution, and tool handlers must return `{ content: [...] }` (the old `{ toolResult }` field was removed from the spec).

Key Findings

1. **Latest stable SDK is v1.29.0.** v1.x is the recommended production line; npm v2.0.0-alpha exists on the `main` branch. Per the typescript-sdk README: *"We anticipate a stable v2 release in Q1 2026… v1.x will continue to receive bug fixes and security updates for at least 6 months after v2 ships."* GitHub
2. **Two server classes exist.** `McpServer` (high-level, `server/mcp.js` , auto-infers capabilities, `registerTool`) and `Server` (low-level, `server/index.js` , explicit `setRequestHandler` + explicit capabilities). For your fixed tool/resource set, either works; the low-level `Server` gives direct control and is what your named request schemas target.
3. **Neo4j driver v5 managed transactions** (`session.executeRead` / `executeWrite`) auto-retry on transient errors with exponential backoff; raw `session.run` is auto-

commit and not retried. `bolt+s://` = encrypted direct; `neo4j+s://` = encrypted with routing.

4. **Neo4j vector indexes** were introduced in Neo4j 5.11 and considered GA with the release of version 5.13 (per Neo4j's Spring AI reference). They are powered by Lucene using a Hierarchical Navigable Small World Graph (HNSW) to perform a k-approximate-nearest-neighbors (k-ANN) query over the vector fields, use cosine/euclidean similarity, and are queried via `db.index.vector.queryNodes`.
5. **Zod version trap**: the SDK has a required peer dependency on `zod`. Per the official README: *"The SDK internally imports from zod/v4, but maintains backwards compatibility with projects using Zod v3.25 or later."* modelcontextprotocol npm An older SDK paired with Zod v4 produced `w._parse is not a function` GitHub errors — pin a compatible pairing.

Details

1. Installation & project setup

```
npm install @modelcontextprotocol/sdk neo4j-driver zod
npm install -D typescript @types/node tsx
```

`package.json` must declare ESM:

```
{
  "type": "module",
  "bin": { "kabbalah-mcp-server": "./dist/index.js" }
}
```

`tsconfig.json` (Node16/NodeNext is required for the `.js`-suffixed subpath imports to resolve):

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "Node16",
    "moduleResolution": "Node16",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true
  }
}
```

2. Server instantiation + request schemas (low-level API, exact import paths)

```
import { Server } from "@modelcontextprotocol/sdk/server/index.js";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";
import {
  ListToolsRequestSchema,
  CallToolRequestSchema,
  ListResourcesRequestSchema,
  ReadResourceRequestSchema,
  ListResourceTemplatesRequestSchema,
} from "@modelcontextprotocol/sdk/types.js";

const server = new Server(
  { name: "kabbalah-mcp-server", version: "1.0.0" },
  { capabilities: { tools: {}, resources: {} } }  // low-level Server:
must declare capabilities
);
```

Method signatures:

- `new Server(serverInfo: {name, version}, options: {capabilities, instructions?})` — for the low-level `Server`, you MUST declare each capability you use (`tools: {}`, `resources: {}`); if you forget one, the client won't know about it and requests for that feature fail.
- `server.setRequestHandler(schema, async (request, extra) => result)` — the handler's request shape and return type are inferred from the Zod request schema.
- `server.connect(transport): Promise<void>`

ListTools handler — note `inputSchema` is raw JSON Schema (not Zod) when using the low-level `Server`:

```
server.setRequestHandler(ListToolsRequestSchema, async () => ({
  tools: [
    {
      name: "resolve_coordinate",
      description: "Resolve a Kabbalistic coordinate
(Gate/Code/Bahir/Zohar) to its node.",
      inputSchema: {
        type: "object",
        properties: { coordinate: { type: "string", description: "e.g.
'Gate:3/Bahir:17'" } },
        required: ["coordinate"],
      },
    },
    {
```

```
      name: "find_intersections",
      description: "Find cross-references between two corpora
coordinates.",
      inputSchema: {
        type: "object",
        properties: {
          source: { type: "string" },
          target: { type: "string" },
        },
        required: ["source", "target"],
      },
    },
    {
      name: "traverse_lineage",
      description: "Traverse the lineage/ancestry graph from a node.",
      inputSchema: {
        type: "object",
        properties: {
          start: { type: "string" },
          depth: { type: "number", description: "max hops" },
        },
        required: ["start"],
      },
    },
    {
      name: "semantic_search",
      description: "Embedding-based semantic search across the
hypermetagraph.",
      inputSchema: {
        type: "object",
        properties: {
          query: { type: "string" },
          topK: { type: "number" },
        },
        required: ["query"],
      },
    },
  ],
}));
```

CallTool handler — return `{ content: [...] }`; report execution errors in-band with `isError: true`:

```
server.setRequestHandler(CallToolRequestSchema, async (request) => {
  const { name, arguments: args } = request.params;
  try {
    switch (name) {
      case "resolve_coordinate":
        return { content: [{ type: "text", text: JSON.stringify(await
resolveCoordinate(args.coordinate)) }] };
```

```
      case "find_intersections":
        return { content: [{ type: "text", text: JSON.stringify(await
findIntersections(args.source, args.target)) }] };
      case "traverse_lineage":
        return { content: [{ type: "text", text: JSON.stringify(await
traverseLineage(args.start, args.depth ?? 3)) }] };
      case "semantic_search":
        return { content: [{ type: "text", text: JSON.stringify(await
semanticSearch(args.query, args.topK ?? 10)) }] };
      default:
        throw new Error(`Unknown tool: ${name}`);
    }
  } catch (err) {
    return { isError: true, content: [{ type: "text", text: `Error:
${(err as Error).message}` }] };
  }
});
```

> Note: In recent 1.x, **unknown tool names** now return a JSON-RPC protocol error (code -32602 InvalidParams) rather than a `CallToolResult` with `isError:true`; unknown resources return -32002 (ResourceNotFound). GitHub Reserve `isError:true` for *execution* errors the model should see and recover from.

3. Resource URI conventions + handlers

MCP resources are read-only, addressed by URI. The spec defines standard schemes (`file://`, `https://`, `git://`) Model Context Protocol but **explicitly permits custom schemes** — *"Custom URI schemes MUST be in accordance with RFC3986."* Model Context Protocol For a Kabbalistic corpus, define schemes like `gate://`, `bahir://`, `zohar://`, `code://`.

```
server.setRequestHandler(ListResourcesRequestSchema, async () => ({
  resources: [
    { uri: "zohar://1:15a", name: "Zohar 1:15a", mimeType: "text/plain",
description: "Zohar folio" },
    { uri: "bahir://17",    name: "Sefer Bahir §17", mimeType:
"text/plain" },
  ],
}));

// Parameterized resources via URI templates (RFC 6570)
server.setRequestHandler(ListResourceTemplatesRequestSchema, async () =>
({
  resourceTemplates: [
    { uriTemplate: "gate://{gate}/{verse}", name: "Seven Gates passage",
mimeType: "text/plain" },
    { uriTemplate: "zohar://{volume}:{folio}", name: "Zohar folio",
mimeType: "text/plain" },
  ],
```

```
}));

server.setRequestHandler(ReadResourceRequestSchema, async (request) => {
  const uri = request.params.uri;
  const m = /^zohar:\/\/(.+)$/.exec(uri);
  if (m) {
    const text = await loadZoharFolio(m[1]);
    return { contents: [{ uri, mimeType: "text/plain", text }] };
  }
  throw new Error(`Resource not found: ${uri}`);
});
```

The `ReadResource` result is `{ contents: [{ uri, mimeType?, text }] }` for text, or `{ uri, mimeType, blob }` (base64) for binary. **Always validate/whitelist the URI** before reading to prevent traversal (e.g. `if (!uri.startsWith("zohar://")) throw new Error("Access denied")`). DEV Community

4. StdioServerTransport connection pattern

```
async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.error("Kabbalah MCP server running on stdio");  // MUST use
stderr
}
main().catch((e) => { console.error(e); process.exit(1); });
```

Critical: never use `console.log` — stdout is the JSON-RPC channel; any stray write corrupts the protocol stream. Use `console.error` for all diagnostics. `new StdioServerTransport()` takes no arguments and reads/writes JSON-RPC over stdin/stdout. This is the standard transport for Claude Desktop and other local, process-spawned integrations.

5. Breaking changes 0.x → 1.x

The last 0.x release was `0.6.1`; v1.0.0 shipped in the MCP launch window (≈ November 2024). For a low-level `Server`:

Change	0.x	1.x
Import paths (biggest break)	`@modelcontextprotocol/sdk/server`, `.../server/stdio` (extensionless)	`@modelcontextprot` `.../server/stdio.` `.js`, ESM NodeNext)
Tool result shape	`{ toolResult: value }`	`{ content: [{ typ` errors `{ isError: t`

Change	0.x	1.x
`McpServer` high-level class	did not exist	added at `server/mcp`
`Server` constructor `{name,version}, {capabilities}`	same	**unchanged**
`setRequestHandler`, `StdioServerTransport`, `server.connect()`	same	**unchanged** (only impo

Net: the low-level `Server` + `setRequestHandler` API is essentially stable across 0→1; the required edits are (a) add `.js` to imports + switch tsconfig to `Node16` / `NodeNext` + `"type":"module"`, and (b) change tool returns from `toolResult` to `content`. The SDK maintainer confirmed the removal: *"I was informed that toolResult is no longer a thing in the spec."* GitHub

Do **not** confuse these with the separate **v2.0.0-alpha** changes, which restructure into a monorepo with new packages (`@modelcontextprotocol/server`, `/node`, `/express`, `/hono`) and remove deprecated methods — confirmed in the 2.0.0-alpha release notes: PR #1419 *"deprecated .tool, .prompt, .resource method removal"* and PR #1279 *"Initial 2.0.0-alpha.0 client and server package."* GitHub

6. claude_desktop_config.json registration

Config file location:

- macOS: `~/Library/Application Support/Claude/claude_desktop_config.json` Natoma
- Windows: `%APPDATA%\Claude\claude_desktop_config.json` Natoma

For a compiled local server:

```
{
  "mcpServers": {
    "kabbalah-hypermetagraph": {
      "command": "node",
      "args": ["/absolute/path/to/kabbalah-mcp-server/dist/index.js"],
      "env": {
        "NEO4J_URI": "neo4j+s://xxxx.databases.neo4j.io",
        "NEO4J_USER": "neo4j",
        "NEO4J_PASSWORD": "your-password",
        "CRED_TRUST_FLOOR": "0.743"
      }
    }
```

```
  }
}
```

For dev (run TS directly without building):

```
{
  "mcpServers": {
    "kabbalah-hypermetagraph": {
      "command": "npx",
      "args": ["tsx", "/absolute/path/to/src/index.ts"]
    }
  }
}
```

Rules: **absolute paths only** (relative paths fail silently); MCPize on Windows escape backslashes (`C:\\Users\\...`); if `node` / `npx` isn't on Claude Desktop's PATH (common with nvm/Homebrew/fnm) use the full binary path (find via `which npx`); Toolradar JSON must be syntactically perfect (a trailing comma silently disables all servers); fully quit and restart Claude Desktop (Cmd+Q on macOS — closing the window is not enough) after editing; Toolradar verify via the hammer/tool icon at the bottom of a new chat; Toolradar check logs at `~/Library/Logs/Claude/mcp-server-*.log` (macOS).

7. Neo4j JavaScript driver v5 — instantiation + transactions

```
import neo4j from "neo4j-driver";

const driver = neo4j.driver(
  process.env.NEO4J_URI!,                          // neo4j+s:// or
bolt+s://
  neo4j.auth.basic(process.env.NEO4J_USER!,
process.env.NEO4J_PASSWORD!),
  {
    maxConnectionPoolSize: 100,
    connectionAcquisitionTimeout: 60_000,          // ms (default 60s)
    connectionTimeout: 30_000,                     // ms (default 30s
since 4.x)
    maxConnectionLifetime: 3_600_000,              // 1 hour (default)
    maxTransactionRetryTime: 30_000,               // default 30s
    disableLosslessIntegers: true,                 // return JS numbers,
not Integer objects
  }
);
```

URI schemes — the scheme is dictated by server config, not preference (*"The connection scheme to use is not your choice, but is rather determined by the server requirements"*): Neo4j

Scheme	Encryption	Routing
`bolt://`	none	direct (single instance)
`bolt+s://`	TLS, full cert check	direct
`bolt+ssc://`	TLS, self-signed cert	direct
`neo4j://`	none	routing (cluster/Aura)
`neo4j+s://`	TLS, full cert check	routing — **use this for Neo4j Aura**
`neo4j+ssc://`	TLS, self-signed cert	routing

Per the docs: *"The +s variants enable encryption with a full certificate check. The +ssc variants enable encryption with no certificate check."* Neo4j Do **not** combine `+s` / `+ssc` URIs with the `encrypted` / `trust` config options — that throws a config conflict; use one mechanism or the other.

Managed transactions (auto-retried) — always pass `database` explicitly and close sessions:

```
// READ
async function resolveCoordinate(coordinate: string) {
  const session = driver.session({ database: "neo4j", defaultAccessMode:
neo4j.session.READ });
  try {
    return await session.executeRead(async (tx) => {
      const res = await tx.run(
        `MATCH (n:Node {coordinate: $coordinate}) RETURN n`,
        { coordinate }
      );
      return res.records.map((r) => r.get("n").properties);  //
materialize INSIDE the txFn
    });
  } finally {
    await session.close();
  }
}

// WRITE (e.g. appending to the TOFAR ledger)
async function appendLedger(entry: object) {
  const session = driver.session({ database: "neo4j" });
  try {
    await session.executeWrite(async (tx) => {
      await tx.run(`CREATE (e:TofarEntry $props) RETURN e`, { props:
entry });
    });
  } finally {
```

```
    await session.close();
  }
}
```

- `session.executeRead(txFn, txConfig?)` / `session.executeWrite(txFn, txConfig?)` — the callback `tx` exposes `tx.run(cypher, params)`. Optional second arg is a `TransactionConfig` `{ timeout, metadata }`. Neo4j These automatically override the session's `defaultAccessMode`. Neo4j
- `session.run(cypher, params)` — auto-commit, **not** retried; Neo4j use only for `CALL {} IN TRANSACTIONS` / `USING PERIODIC COMMIT`, which managed transactions reject. Go Packages
- `driver.executeQuery(cypher, params, { database, routing: neo4j.routing.READ })` — newest high-level one-shot API; wraps a managed transaction. Always pass `database` to avoid an extra round-trip resolving the home DB. Neo4j
- **Never** return the raw `Result` object out of a `txFn`; always materialize (`.records.map(...)`) inside.
- Call `driver.close()` and `session.close()` on shutdown or the Node process may hang on exit. Neo4j

8. Neo4j vector index (semantic_search)

Create the index (cosine is best for text embeddings; `vector.dimensions` must match your embedding model — 1536 for OpenAI `text-embedding-3-small`, 768 for many sentence-transformers):

```
CREATE VECTOR INDEX kabbalah_embeddings IF NOT EXISTS
FOR (n:Passage) ON (n.embedding)
OPTIONS {indexConfig: {
  `vector.dimensions`: 1536,
  `vector.similarity_function`: 'cosine'
}}
```

The backticks around `vector.dimensions` / `vector.similarity_function` are required — they are dotted config keys. `IF NOT EXISTS` makes startup creation idempotent. Neo4j Valid similarity functions are `'cosine'` and `'euclidean'`. Neo4j

Store embeddings with the validated procedure (do not just `SET n.embedding = [...]`):

```
MATCH (n:Passage {id: $id})
CALL db.create.setNodeVectorProperty(n, 'embedding', $vector)
```

Signature: `db.create.setNodeVectorProperty(node :: NODE, key :: STRING, vector :: ANY)`. Neo4j It validates the property is a valid vector.

Query from the JS driver:

```
async function semanticSearch(query: string, topK = 10) {
  const queryVector = await embed(query);          // your embedding
call -> number[]
  const session = driver.session({ database: "neo4j", defaultAccessMode:
neo4j.session.READ });
  try {
    return await session.executeRead(async (tx) => {
      const res = await tx.run(
        `CALL db.index.vector.queryNodes('kabbalah_embeddings', $topK,
$queryVector)
         YIELD node, score
         RETURN node.id AS id, node.text AS text, score
         ORDER BY score DESC`,
        { topK: neo4j.int(topK), queryVector }
      );
      return res.records.map((r) => ({ id: r.get("id"), text:
r.get("text"), score: r.get("score") }));
    });
  } finally {
    await session.close();
  }
}
```

`db.index.vector.queryNodes(indexName :: STRING, numberOfNearestNeighbours :: INTEGER, query :: ANY)` Neo4j returns `(node, score)` with scores bounded in [0,1] (closer to 1 = more similar). Neo4j Pass `topK` as `neo4j.int(...)` since the procedure expects an INTEGER. Under the hood the index is a Lucene HNSW graph performing approximate k-NN.

> Version note: as of Neo4j 2025.x there is a newer native `VECTOR` type (per the 2025 changelog: *"Introduced VECTOR as a new type supported by Cypher, Block store format, and v6 drivers"*; GitHub the `vector-3.0` provider was introduced in Neo4j 2025.09) Neo4j and a Cypher `SEARCH ... IN (VECTOR INDEX ...)` clause. Neo4j `db.index.vector.queryNodes` still works and is the broadly compatible choice for 5.x. If you adopt the v6 driver + Cypher 25, prefer `neo4j.vector(Float32Array.from(arr))` to avoid per-element type-tagging overhead on the wire.

9. Integrating the broader system (TOFAR / CRED / Caputo / tekufah)

These are domain concerns to implement *inside* your tool handlers, not MCP/Neo4j primitives:

- **TOFAR append-only ledger** → model as `:TofarEntry` nodes written only via `executeWrite` with `CREATE` (never `MERGE` / `SET` on existing rows), plus a

monotonic sequence number and a prev-hash property to enforce append-only/tamper-evident semantics at the application layer.

- **CRED economy (trust_floor = 0.743)** → enforce in the CallTool handler: reject or down-rank results whose trust score < 0.743 before returning `content`. Inject the floor via env (`CRED_TRUST_FLOOR`) rather than hard-coding.
- **Caputo fractional memory / tekufah cycle tracking** → compute in handler logic; expose either as tool outputs or as read-only resources (e.g. `tekufah://current`) so a client can pull the current cycle state without invoking a tool.

Recommendations

1. **Start now on `@modelcontextprotocol/sdk@^1.29.0` with the low-level `Server` API + stdio transport.** Pin `zod@^3.25` (or import `zod/v4` consistently across your code) to avoid the `_parse` incompatibility. Use Node ≥18, `"type":"module"`, tsconfig `module` / `moduleResolution` = `Node16`. Wire all four tools through one `CallToolRequestSchema` switch and your custom-scheme resources through `ReadResourceRequestSchema`.
2. **Use `neo4j-driver@^5` with `neo4j+s://` for Aura** (or `bolt+s://` for a single self-managed TLS instance), `executeRead` / `executeWrite` everywhere, `disableLosslessIntegers: true`, and always pass `database: "neo4j"` + call `session.close()` in `finally`.
3. **Create the vector index once at startup** (`CREATE VECTOR INDEX ... IF NOT EXISTS`), store embeddings via `db.create.setNodeVectorProperty`, query via `db.index.vector.queryNodes`, and match `vector.dimensions` exactly to your embedding model or inserts will fail.
4. **Register in Claude Desktop with absolute paths**, secrets in `env`, then verify via the tool/hammer icon; debug interactively with `npx @modelcontextprotocol/inspector node dist/index.js` DEV Community before wiring into Claude.
5. **Thresholds that change the plan**: if you need a *remote* multi-client server, switch transport to `StreamableHTTPServerTransport` (not stdio) and add session handling. If you upgrade to Neo4j 2025.x+/Cypher 25 + driver v6, migrate to the native `VECTOR` type, property-type constraints (`REQUIRE x IS :: VECTOR<FLOAT32>(1536)`), and the `SEARCH ... IN (VECTOR INDEX ...)` clause. If you later need dynamic, per-request tool schemas with automatic JSON-Schema conversion and validation, migrate to `McpServer.registerTool` with Zod input/output schemas.

Caveats

- **Exact v1.0.0 publish date is unverified** from a primary source — GitHub's original 1.0.0 release notes are no longer rendered (the Releases page is dominated by v2.0.0-alpha changesets). Best estimate ≈ Nov 2024; last 0.x = 0.6.1. Treat the date as approximate.
- **`structuredContent` / `outputSchema`** is a post-1.0 addition. If you use it, the SDK validates `structuredContent` first, which can mask an `isError:true` return (known issue #654) GitHub — return both `content` and `structuredContent`, and for backward compatibility a tool returning structured content SHOULD also serialize it into a `TextContent` block.
- **Deprecation horizon for the vector procedure**: Neo4j docs note `db.index.vector.queryNodes` / `queryRelationships` are deprecated as of Neo4j 2026.04 Neo4j in favor of the Cypher `SEARCH` clause, but they remain functional and are the compatible choice for current 5.x deployments — plan a migration path rather than rewriting now.
- **Zod v4 + older SDK** combinations are known to break tool execution (`w._parse is not a function`); validate your exact `@modelcontextprotocol/sdk` ↔ `zod` version pairing before deploying.
- **The TOFAR / CRED / Caputo-fractional-memory / tekufah components are bespoke to your system** and have no standard library or published-spec support that surfaced in research. The architectural guidance above (append-only via `CREATE` + hash chain, trust-floor gating in handlers, cycle state as resources) is sound MCP/Neo4j practice but is not verified against any external specification for those concepts.